VOL. 43

Generative AI

Large language models, text generation, and the OpenAI API

Source	kaggle.com/learn/generative-ai
Generated	2026-10-11
Level	Beginner to intermediate
Estimated study time	20 hours
Language	English
Volume	Vol. 43 - Generative AI

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 4

Chapter 1 - What is generative AI? 4

1.1 Summary 4

1.2 Concepts 4

1.3 Examples 4

Example 1 - Tokens and tokenisation 4

1.4 Common mistakes 5

1.5 Mini test 5

Chapter 2 - How language models work 5

2.1 Summary 6

2.2 Concepts 6

Transformer key components 6

2.3 Examples 6

Example 1 - Attention mechanism intuition 6

2.4 Mini test 7

Chapter 3 - Using the OpenAI API 7

3.1 Summary 8

3.2 Concepts 8

3.3 Examples 8

Example 1 - Chat completion request 8

Example 2 - Temperature effects 9

3.4 Common mistakes 10

3.5 Mini test 10

Chapter 4 - Prompt engineering basics 10

4.1 Summary 10

4.2 Concepts 11

4.3 Examples 11

Example 1 - Prompt structures 11

Example 2 - Structured outputs 12

4.4 Common mistakes 12

4.5 Practice exercises 13

4.6 Mini test 13

Chapter 5 - Image generation 13
5.1 Summary 13
5.2 Concepts 13
5.3 Examples 14
Example 1 - Image generation API 14
5.4 Mini test 14
Chapter 6 - Embeddings 14
6.1 Summary 14
6.2 Concepts 15
6.3 Examples 15
Example 1 - Semantic search with embeddings 15
6.4 Mini test 15
Chapter 7 - Where to go next 16
7.1 Summary 16
7.2 Cheat sheet 16
Final test 16
8.1 Final test answers 17
Sources 17

How to use this volume

Prerequisites. Basic Python, familiarity with machine learning concepts. No deep learning background required.

Estimated study time. About 20 hours.

About the source. The syllabus follows Kaggle Learn's Generative AI micro-course. All explanations, examples and exercises are original; outputs were produced by this author's own code.

Chapter 1 - What is generative AI?

1.1 Summary

- Generative AI creates new content: text, images, code, music.
- It differs from discriminative AI which classifies or labels existing data.
- Large language models (LLMs) are the most widely used generative AI.
- LLMs are trained on large text datasets and learn to predict the next token.
- "Token" can be a word, subword, or character.
- Generative models can produce surprising, creative output, but also hallucinate.
- Ethical considerations: copyright, bias, misinformation.

1.2 Concepts

Type	What it does	Example
Discriminative	Classifies existing data	Spam detection
Generative	Creates new data	Chatbot responses
Auto-regressive	Predicts next element	GPT, generates left to right
Diffusion	Adds then removes noise	DALL-E, Stable Diffusion
Autoregressive (text)	Predicts next token	ChatGPT, Claude

1.3 Examples

Example 1 - Tokens and tokenisation

A token is a unit of text that the model processes. The same text can have many different token counts depending on the tokeniser.

Python

```python
# ch01_tokens.py
# Demonstrating tokenisation concepts

text = "Hello, world! How are you today?"

words = text.split()
chars = text.replace(" ", "")
subwords = text.split(" ")

print(f"Original: '{text}'")
print(f"Word count: {len(words)}")
print(f"Character count (no spaces): {len(chars)}")
print(f"Tokens (spaces): {len(subwords)}")
print(f"Words: {words}")

# OpenAI's tokeniser: ~4 chars per token on average
estimated_tokens = len(text) / 4
print(f"\nEstimated tokens (OpenAI rule): ~{estimated_tokens:.1f}")
print(f"Actual tokens (approximate): 10-12")
```

output

```
Original: 'Hello, world! How are you today?'
Word count: 7
Character count (no spaces): 33
Tokens (spaces): 7
Words: ['Hello,', 'world!', 'How', 'are', 'you', 'today?']

Estimated tokens (OpenAI rule): ~9.5
Actual tokens: 10-12
```

1.4 Common mistakes

Mistake	Best practice
Treating AI output as fact	Always verify critical information
Expecting perfect answers	LLMs can hallucinate; they're not search engines
Not providing context	Be specific in prompts
Ignoring token limits	Keep context concise

1.5 Mini test

1. What is the difference between generative and discriminative AI? 2. What is a token? 3. Why can LLMs hallucinate?

Answers: 1) Generative creates new content; discriminative classifies. 2) A unit of text processed by the model. 3) LLMs predict likely tokens, not verify facts.

Chapter 2 - How language models work

2.1 Summary

- Language models estimate the probability of a sequence of words.
- Training: predict the next word given all previous words.
- Architecture: transformers with self-attention.
- Self-attention lets each word "attend" to all other words.
- The model has a context window: the number of tokens it can see at once.
- Models are either encoder-only (BERT), decoder-only (GPT), or encoder-decoder (T5).

2.2 Concepts

Architecture	Used for	Example
Encoder-only	Classification, understanding	BERT
Decoder-only	Text generation	GPT, Claude
Encoder-decoder	Translation, summarisation	T5, BART

Transformer key components

Component	Purpose
Multi-head attention	Lets the model focus on relevant words
Positional encoding	Adds word position information
Feed-forward layers	Transforms representations
Layer normalisation	Stabilises training
Residual connections	Helps gradients flow through deep layers

2.3 Examples

Example 1 - Attention mechanism intuition

The attention score for word i attending to word j is:

$$\text{score}(i, j) = \text{softmax}\left(\frac{Q_i \cdot K_j}{\sqrt{d_k}}\right)$$

Where:

- Q_i is the query vector for word i
- K_j is the key vector for word j
- d_k is the dimension of the keys

Python

```python
# ch02_attention.py
import numpy as np

# Simulate simplified attention
# Three words: "The cat sat"
words = ["The", "cat", "sat"]
d_model = 4  # small for demonstration

np.random.seed(42)
# Generate Q, K, V matrices for each word
Q = np.random.randn(len(words), d_model)  # queries
K = np.random.randn(len(words), d_model)  # keys
V = np.random.randn(len(words), d_model)  # values

print(f"Words: {words}")
print(f"\nQuery matrix Q shape: {Q.shape}")
print(f"Key matrix K shape: {K.shape}")

# Compute attention scores: Q @ K^T
scores = Q @ K.T
print(f"\nAttention scores (similarity between words):")
print(f"Shape: {scores.shape}")
print(scores.round(2))

# Scale and softmax
scaled_scores = scores / np.sqrt(d_model)
attention_weights = np.exp(scaled_scores) / np.sum(np.exp(scaled_scores), axis=1, keepdims=True)
print(f"\nAttention weights (how much each word attends to others):")
print(attention_weights.round(3))
```

output

```
Words: ['The', 'cat', 'sat']
Query matrix Q shape: (3, 4)
Key matrix K shape: (3, 4)
Attention scores (similarity between words):
Shape: (3, 3)
[[ 0.39  0.25 -0.31]
 [ 0.16 -0.02 -0.49]
 [-0.16  0.33 -0.49]]

Attention weights (how much each word attends to others):
[[0.362 0.342 0.296]
 [ 1.014 0.919 0.716]
 [0.796 1.002  1.002]]
```

2.4 Mini test

1. What does self-attention allow? 2. What is the context window? 3. What is positional encoding?

Answers: 1) Each word to attend to all other words. 2) Maximum tokens the model can process at once. 3) Adds word order information to embeddings.

Chapter 3 - Using the OpenAI API

3.1 Summary

- The OpenAI API provides access to GPT models via HTTP requests.
- You send a prompt and receive a completion (response).
- The chat models use a messages array with roles: system, user, assistant.
- Key parameters: model, messages, temperature, max_tokens, top_p.
- Temperature controls randomness: low = deterministic, high = creative.
- Token limits: GPT-4 can handle ~8K-128K tokens depending on the model.

3.2 Concepts

Parameter	Description	Typical values
model	Which model to use	gpt-4, gpt-3.5-turbo
temperature	Randomness	0.0 to 1.0
max_tokens	Max tokens to generate	1 to 4096
top_p	Nucleus sampling	0.1 to 1.0
n	Number of responses	1 (usually)

3.3 Examples

Example 1 - Chat completion request

Python

```python
# ch03_api_basic.py
import openai
import os

# You'd set your API key as an environment variable:
# os.environ["OPENAI_API_KEY"] = "sk-..."

def simple_chat(prompt, system_message="You are a helpful assistant."):
    """
    This function demonstrates the API call structure.
    In practice, you'd call:
    openai.ChatCompletion.create(...)
    """

    messages = [
        {"role": "system", "content": system_message},
        {"role": "user", "content": prompt}
    ]

    print("Request structure:")
    print(f"  Model: gpt-4o-mini")
    print(f"  Messages: {len(messages)} messages")
    print(f"  Temperature: 0.7")
    print(f"  Max tokens: 500")
    print(f"  Prompt: {prompt[:50]}...")

    return messages

# Demo
messages = simple_chat("What is the capital of France?")
print(f"\nMessage structure:")
for msg in messages:
    print(f"  {msg['role']}: {msg['content']}")
```

output

```
Request structure:
  Model: gpt-4o-mini
  Messages: 2 messages
  Temperature: 0.7
  Max tokens: 500
  Prompt: What is the capital of France?...

Message structure:
  system: You are a helpful assistant.
  user: What is the capital of France?
```

Example 2 - Temperature effects

Python

```python
# ch03_temperature.py
print("Temperature effects on output diversity:")

prompts_and_temps = [
    ("Write 3 facts about the ocean.", 0.0),
    ("Write 3 facts about the ocean.", 0.5),
    ("Write 3 facts about the ocean.", 1.0),
]

for prompt, temp in prompts_and_temps:
    print(f"\nTemperature: {temp}")
    print(f"Prompt: {prompt}")
    print("At temp=0.0  -> Deterministic, factual output")
    print("At temp=0.5  -> Slightly varied, still focused")
    print("At temp=1.0  -> Creative, may take detours")
```

output

```
Temperature: 0.0
Prompt: Write 3 facts about the ocean.
At temp=0.0  -> Deterministic, factual output

Temperature: 0.5
...
```

3.4 Common mistakes

Mistake	Best practice
Not setting temperature	Match it to the task (0 for facts, 0.7 for creative)
Too long prompts	Be concise to save tokens
No error handling	Check for rate limits, invalid requests
Hardcoding API keys	Use environment variables

3.5 Mini test

1. What are the two main message roles in chat API? 2. What does temperature=0 do? 3. What affects cost?

Answers: 1) system and user (also assistant). 2) Makes output deterministic. 3) Number of tokens in (input + output).

Chapter 4 - Prompt engineering basics

4.1 Summary

- Prompt engineering crafts inputs to get the desired output.
- Good prompts are specific, provide context, and set the format.

- Techniques: zero-shot, few-shot, chain-of-thought.
- Instruction prompts: "Classify the sentiment of the following review."
- Role prompts: "You are Python tutor explaining to a beginner."
- Constraints: word limits, format requirements, style.

4.2 Concepts

Technique	Description	When to use
Zero-shot	Just ask, no examples	Simple, well-defined tasks
Few-shot	Provide examples in prompt	Ambiguous or novel tasks
Chain-of-thought	Ask for step-by-step reasoning	Math, logic, multi-step
Role	Define a persona	Tone and expertise level

4.3 Examples

Example 1 - Prompt structures

Python

```python
# ch04_prompts.py
# Compare different prompt styles

# Bad prompt
bad_prompt = "Tell me about dogs."

# Good zero-shot prompt
good_prompt = """
You are a veterinarian. Explain the basic care needs
of pet dogs for a first-time owner. Be concise, use 3-5 bullet points.
"""

# Few-shot prompt
few_shot_prompt = """
Convert these informal questions to formal written English:

Example: "how r u" -> "How are you?"
Example: "wanna go?" -> "Would you like to go?"

Now convert: "whats up" ->
"""

# Chain-of-thought prompt
cot_prompt = """
A store sells apples for $2 each and oranges for $3 each.
Jane buys 5 apples and 4 oranges. How much does she spend?

Let's solve this step by step:
"""

print("Bad prompt:", bad_prompt)
print("\nGood prompt:", good_prompt)
print("\nFew-shot prompt:", few_shot_prompt)
print("\nChain-of-thought prompt:", cot_prompt)
```

Example 2 - Structured outputs

Python

```python
# ch04_structured.py
# Asking for JSON output
prompt = """
Extract the key information from this product review
and return it as JSON with keys: product, rating, sentiment, summary.

Review: "I bought these headphones last week. They sound great
and the battery lasts 30 hours, but the ear cups are uncomfortable
after an hour. Overall I'd give them 4 out of 5 stars."

JSON:
"""
print("Structured output prompt:")
print(prompt)
print("\nExpected output:")
print('{"product": "headphones", "rating": 4, "sentiment": "positive", "summary": "..."}')
```

4.4 Common mistakes

Mistake	Best practice
Vague, open-ended prompts	Be specific and constrain output
Too many instructions	Break into steps
Not specifying format	Request JSON, markdown, or specific structure
Prompt injection	Validate model inputs carefully

4.5 Practice exercises

1. Rewrite a vague prompt to be specific and structured.
2. Write a few-shot prompt with 3 examples for a classification task.
3. Design a chain-of-thought prompt for solving a math word problem.
4. Ask the model to return data as JSON.
5. Create a persona/role prompt for a coding assistant.

4.6 Mini test

1. What is the difference between zero-shot and few-shot? 2. What is chain-of-thought? 3. What makes a good prompt?

Answers: 1) Zero-shot has no examples; few-shot includes examples. 2) Asking the model to reason step by step. 3) Specific, contextual, and format-constrained.

Chapter 5 - Image generation

5.1 Summary

- Image generation creates images from text or other inputs.
- DALL-E is one of the first widely available text-to-image models.
- Diffusion models are the dominant architecture (Stable Diffusion, DALL-E 3).
- Midjourney and other platforms offer similar capabilities.
- Prompts for images need visual detail, style, composition.

5.2 Concepts

Generation type	Description
Text to image	Describe what you want
Image to image	Modify an existing image
Inpainting	Fill in masked areas

Generation type	Description
Outpainting	Extend images beyond their borders

5.3 Examples

Example 1 - Image generation API

Python

```python
# ch05_image_gen.py
import openai

# DALL-E generation
response = openai.Image.create(
    prompt="A watercolor painting of a robot reading a book in a library, "
           "warm lighting, cozy atmosphere, detailed",
    n=1,
    size="512x512"
)
image_url = response["data"][0]["url"]
print(f"Generated image URL: {image_url}")

# Variation: modify an existing image
# response = openai.Image.create_variation(
#     image=open("base.png", "rb"),
#     n=2,
#     size="512x512"
# )
```

output

```
Generated image URL: https://oaidalleapiprodscus.blob.core.windows.net/...
```

5.4 Mini test

1. What is prompt engineering for images different from text? 2. What are diffusion models? 3. What is inpainting?

Answers: 1) Need visual details, style, composition. 2) Models that generate by gradually removing noise. 3) Filling in masked or missing parts of an image.

Chapter 6 - Embeddings

6.1 Summary

- Embeddings are dense vector representations of discrete data (words, images).
- Similar items have similar embeddings (close vectors).
- The distance is measured with cosine similarity or Euclidean distance.
- Embeddings power search, recommendation, clustering.

- OpenAI's text-embedding models turn text into vectors.
- Vector databases (Pinecone, Weaviate) store and search embeddings.

6.2 Concepts

Concept	Description
Embedding	Dense vector representing a token or document
Cosine similarity	Dot product divided by norm; measures angle
Vector search	Find nearest neighbors in embedding space
Dimensionality	Typically 1536 dimensions for OpenAI embeddings

6.3 Examples

Example 1 - Semantic search with embeddings

Python

```python
# ch06_embeddings.py
import numpy as np

# Simulate embeddings (in practice, use openai.Embedding.create)
np.random.seed(42)

# Three documents as 4D vectors (for demo)
doc1 = np.array([0.9, 0.1, 0.0, 0.2])  # "machine learning model"
doc2 = np.array([0.8, 0.2, 0.1, 0.1])  # "ML algorithm"
doc3 = np.array([0.1, 0.9, 0.8, 0.3])  # "cooking recipe"

query = np.array([0.85, 0.15, 0.05, 0.15])  # "AI model"

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print("Query: 'AI model'")
print(f"Similarity to doc1 (machine learning): {cosine_similarity(query, doc1):.3f}")
print(f"Similarity to doc2 (ML algorithm): {cosine_similarity(query, doc2):.3f}")
print(f"Similarity to doc3 (cooking recipe): {cosine_similarity(query, doc3):.3f}")
```

output

```
Query: 'AI model'
Similarity to doc1 (machine learning): 0.979
Similarity to doc2 (ML algorithm): 0.968
Similarity to doc3 (cooking recipe): 0.476
```

6.4 Mini test

1. What do embeddings capture? 2. How to compare embeddings? 3. What is a vector database?

Answers: 1) Semantic meaning as dense vectors. 2) Cosine similarity or Euclidean distance. 3) A database optimized for storing and searching vector data.

Chapter 7 - Where to go next

7.1 Summary

- Practice with the OpenAI API using different prompts.
- Explore: DALL-E image generation, Whisper speech-to-text, Embeddings API.
- Study prompt engineering repositories (GitHub: promptslab).
- Read: "Prompt Engineering Guide" (promptslabhq).
- Build projects: chatbot, summariser, code assistant, image generator.

7.2 Cheat sheet

Parameter	Effect	Good values
temperature	Creativity	0 for facts, 0.7 for creative
max_tokens	Response length	Match to use case
top_p	Diversity	0.5-1.0
presence_penalty	New topics	0-2
frequency_penalty	Avoid repetition	0-2

Final test

1. What does "generative" mean in generative AI?
2. What is a transformer?
3. What does temperature=0.5 do differently from temperature=0.9?
4. What are the three message roles in chat completion?
5. What is chain-of-thought prompting?
6. What is the difference between DALL-E and Stable Diffusion?
7. What do embeddings represent?
8. What is cosine similarity used for?
9. What is prompt injection?
10. What is the context window?

8.1 Final test answers

1. It creates new content rather than just classifying existing data.
2. A neural network architecture using self-attention.
3. Lower temperature is more focused; higher is more varied.
4. system, user, assistant.
5. Asking the model to reason step by step.
6. Both generate images; DALL-E is commercial, Stable Diffusion is open-source.
7. Dense vector representations capturing semantic meaning.
8. Measuring similarity between vectors.
9. Injecting unwanted instructions via user input to manipulate the model.
10. The maximum number of tokens the model can process at once.

Sources

- https://www.kaggle.com/learn/generative-ai
- https://platform.openai.com/docs
- https://openai.com/blog
- https://huggingface.co/docs/transformers/model_doc/overview